

The logo for Stacja.IT features a large, stylized number '6' in the background. The '6' is composed of a thick yellow outer ring and a blue inner ring. In the center of the '6' is a white circle containing the text 'STACJA.IT' in bold, black, uppercase letters.

STACJA.IT

Zabezpieczenie aplikacji frontend i backend

Patryk Tokarz

Poznaj Stację IT



Doświadczenie szkoleniowe

Organizujemy warsztaty IT od 2015 roku, a naszą misją jest przekazywanie porządnej dawki praktycznej wiedzy i umożliwienie bezpośredniego kontaktu z daną technologią.



Szeroki obszar działania

Stanowimy część czegoś większego i jesteśmy jedną z marek Sages – firmy z 17-letnim doświadczeniem, oferującej m.in. szkolenia IT, oprogramowanie dla uczelni czy też chatboty „szyte na miarę”.



Stacja IT w liczbach

Współpracujemy z ponad 300 trenerami, a w ramach samych warsztatów Stacji IT zorganizowaliśmy więcej niż 1 000 wydarzeń i przeszkoliliśmy ponad 12 000 osób.

PARTNERZY

sages

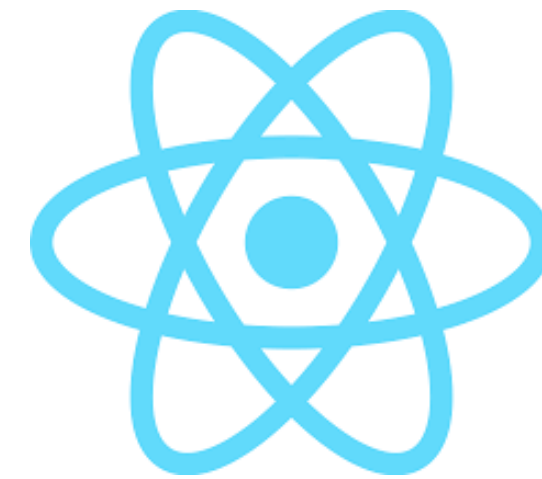
kodo/amacz
by sages



www.stacja.it

Vistula Bank

- React
- Express.js
- SQLite



ex



Przepływ żądań

Formularz React → API → route →
controller → service → model →
SQLite

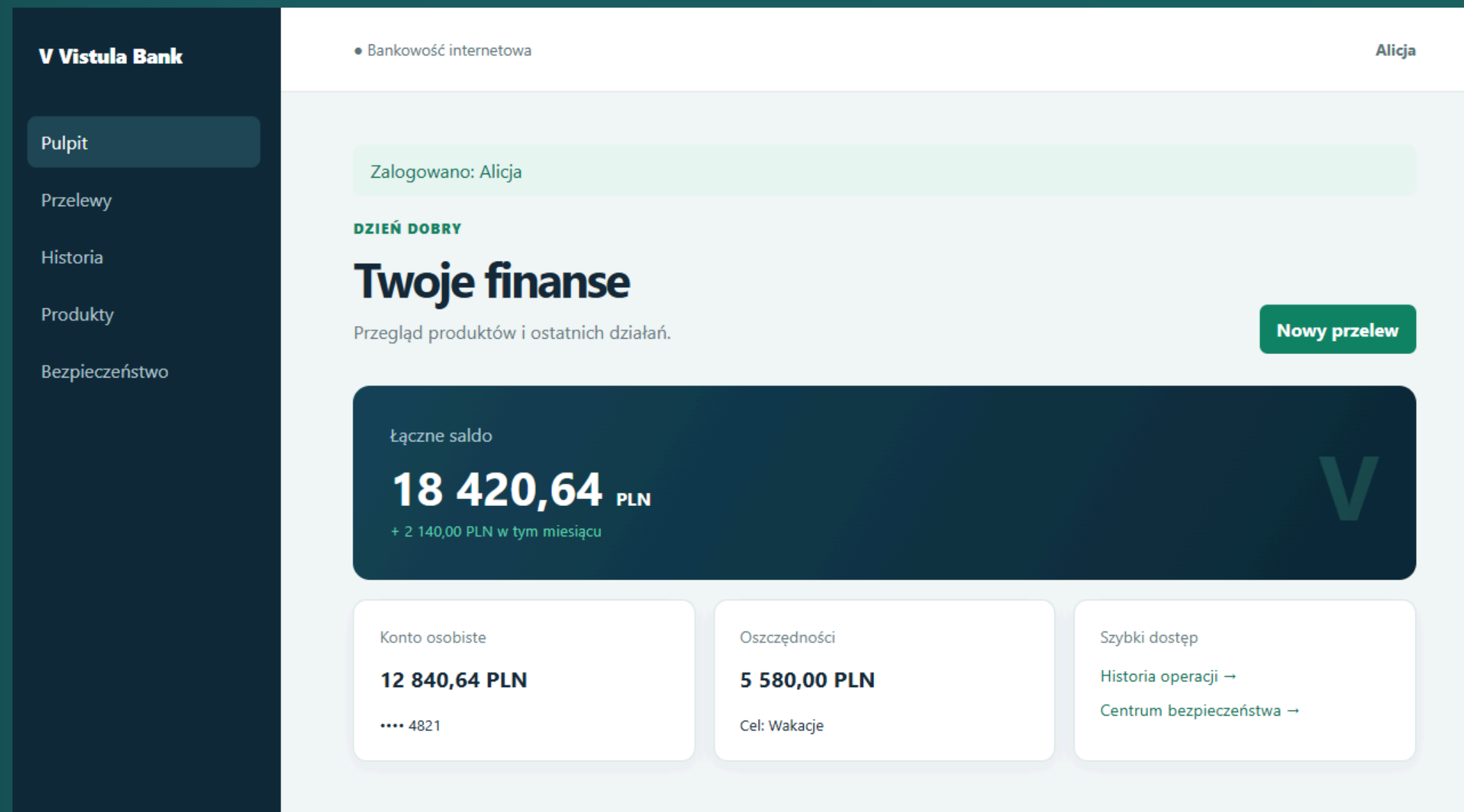
```
projekt/
├── client/                                # aplikacja React, dostępna na porcie 5173
│   ├── src/
│   │   ├── main.jsx                      # punkt wejścia Reacta
│   │   ├── app/App.jsx                  # główny komponent i przełączanie widoków po hashu URL
│   │   ├── pages/                       # widoki panelu bankowego
│   │   │   ├── DashboardPage.jsx        # pulpit i przykładowe saldo
│   │   │   ├── TransfersPage.jsx        # formularz przelewu
│   │   │   ├── ActivityPage.jsx         # historia dyspozycji z API
│   │   │   ├── ProductsPage.jsx         # statyczna oferta banku
│   │   │   └── SecurityPage.jsx         # widok ustawień bezpieczeństwa
│   │   ├── features/auth/components/
│   │   │   ├── LoginForm.jsx            # formularz logowania
│   │   │   └── RegisterForm.jsx         # formularz rejestracji
│   │   ├── services/api.client.js       # wszystkie wywołania HTTP do Express
│   │   ├── config/api.config.js         # adres API: http://localhost:3001/api
│   │   └── styles/global.css            # wygląd aplikacji
│   ├── vite.config.js                   # konfiguracja serwera Vite
│   └── package.json                     # skrypty i zależności frontendu
├── server/                              # API Express, dostępne na porcie 3001
│   ├── server.js                        # start serwera i inicjalizacja bazy
│   └── src/
│       ├── app.js                       # middleware Express i rejestracja tras API
│       ├── config/database.js           # połączenie z SQLite i inicjalizacja tabel
│       ├── routes/                      # auth.routes.js, posts.routes.js, debug.routes.js, health.routes.js
│       ├── controllers/                 # auth, posts, debug i health
│       ├── services/                    # logika logowania, rejestracji i dyspozycji
│       ├── models/                      # user.model.js i post.model.js: zapytania SQL
│       └── middleware/error-handler.middleware.js # globalna obsługa błędów
├── data/workshop.db                     # lokalna baza SQLite, tworzona przy pierwszym uruchomieniu
├── README.md                            # szybka instrukcja uruchomienia projektu
├── tutorial.md                           # notatki prowadzącego do tego warsztatu
└── package.json                          # wspólny skrypt npm run dev
```

Czy aplikacja jest już u Ciebie uruchomiona?

Uruchomienie projektu:

```
npm run install:all  
npm run dev
```

<http://localhost:5173>



Walidacja danych wejściowych: pierwsza brama API

Dane z przeglądarki nie są zaufane.

Źródła danych:

Formularz React

Żądanie HTTP

Przeglądarka



API



Baza

Frontend nie wystarcza

Rejestracja

Nazwa wyświetlana

E-mail

Hasło

Utwórz konto

Wersja startowa celowo nie waliduje tych pól.

Walidacja React

- lepszy komunikat
- mniej błędnych formularzy
- można ją ominąć

Walidacja API

- chroni bazę
- działa dla każdego klienta
- jest obowiązkowa

Frontend poprawia UX. Backend egzekwuje reguły.

Atakuj!

Rejestracja

Nazwa wyświetlana

E-mail

Hasło

Utwórz konto

Wersja startowa celowo nie waliduje tych pól.

1. Otwórz aplikację: <http://localhost:5173/#/access>.
2. W formularzu Rejestracja wpisz:

Nazwa wyświetlana: testowy
E-mail: nie-jest-emailem-01
Hasło: abcd

3. Kliknij Utwórz konto.
4. Otwórz w nowej karcie: <http://localhost:3001/api/debug/users>

Reguły zapisujemy jako schemat Yup

Yup to biblioteka JavaScript do deklaratywnego opisywania reguł danych i sprawdzania, czy obiekt im odpowiada.

- opisujemy reguły w jednym schemacie;
- walidujemy cały obiekt formularza;
- otrzymujemy listę konkretnych błędów pól.

Yup nie zastępuje walidacji backendu.
Może być użyty zarówno w React, jak i w Express.

```
const registerSchema = yup.object({  
  displayName: yup.string().required().max(60),  
  email: yup.string().email().required(),  
  password: yup.string().min(12).required(),  
});
```



Naprawiamy problem - gdzie zmierzamy?

Klient:

- sprawdza nazwę: 2–60 znaków;
- sprawdza format e-maila;
- wymaga hasła o długości 12–128 znaków;
- wyświetla błąd bezpośrednio pod konkretnym polem.

Serwer:

- waliduje POST /api/auth/register niezależnie od Reacta;
- odrzuca błędne dane odpowiedzią 400;
- zwraca komunikaty przypisane do pól;
- usuwa nieznane pola z żądania przed przekazaniem go dalej.

```
const registerSchema = yup.object({
  displayName: yup.string().required().max(60),
  email: yup.string().email().required(),
  password: yup.string().min(12).required(),
});
```



SQL Injection: gdy dane stają się kodem

SQL Injection występuje wtedy, gdy tekst wpisany przez użytkownika jest doklejany do zapytania SQL.

Kod SQL i dane powinny być rozdzielone.
W wersji startowej są jednym ciągiem tekstu.

```
const user = db.prepare(`
  SELECT id, email
  FROM users
  WHERE email = '${email}'
  AND password = '${password}'
`).get();
```

prepare() nie zabezpiecza aplikacji, jeśli wcześniej zbudowaliśmy SQL przez interpolację tekstu.

Atak na logowanie Vistula Bank

Logowanie

E-mail

Hasło

Zaloguj się

Spacja po -- jest częścią komentarza SQL w SQLite.

```
SELECT id, email, display_name
FROM users
WHERE email = '' OR 1=1 -- '
AND password = 'dowolne'
```

Warunek hasła przestaje mieć znaczenie. API zwraca pierwszego użytkownika pasującego do zapytania.

To nie jest tylko problem logowania

Logowanie

Payload w polu e-mail:
' OR 1=1 --

Skutek:
warunek hasła zostaje
zakomentowany.

Wyszukiwanie

Payload w parametrze q (adres):
' OR 1=1 --

Skutek:
zamiast wyników dla jednej frazy
API zwraca wszystkie wpisy.

Rejestracja

Dodajmy użytkownika:

Nazwa wyświetlana: Ała
E-mail: test', 'hasło-atakujacego', 'Administrator') --
Hasło: dowolne

```
INSERT INTO users (email, password, display_name) VALUES ('test', 'hasło-atakujacego', 'Administrator') -- ',  
'dowolne', 'Ała')
```

Poprawka: parametryzowane zapytania

Przed podatne

```
db.prepare(`
  SELECT id, email, display_name
  FROM users
  WHERE email = '${email}'
        AND password = '${password}'
`).get();
```

Dane zmieniają składnię SQL.

Wyszukiwanie

```
db.prepare(`
  SELECT id, email, password, display_name
  FROM users
  WHERE email = ?
`).get(email);
```

Hasło porównujemy później przez `bcrypt.compare()`.
Nie porównujemy go w SQL.

Baza otrzymuje kod SQL i dane jako dwa oddzielne elementy. Apostrof w e-mailu jest danymi, a nie instrukcją dla bazy.

Hasło nie powinno trafiać do bazy wprost

Wersja startowa:

hasło ➤ baza

Wersja poprawna:

hasło ➤ bcrypt ➤ hash ➤ baza

Baza nie musi znać oryginalnego hasła po rejestracji.

Wyciek bazy = wyciek haseł

```
[
  {
    "email": "alice@example.test",
    "password": "alice123",
    "display_name": "Alicja"
  }
]
```

Atakujący może:

- zalogować się jako użytkownik;
- sprawdzić użycie hasła w innych serwisach;
- wykorzystać konto do dalszych ataków.

Hasło jawne jest gotowe do użycia natychmiast po wycieku.

Hash to nie szyfrowanie

Hasło jawne	Szyfrowanie	Hash hasła
można je od razu odczytać	można odszyfrować kluczem	nie odzyskujemy hasła
nie nadaje się do bazy	służy do ochrony danych	służy do weryfikacji hasła
alice123	...	\$2b\$12\$...

bcrypt dodaje salt i wykonuje kosztowne obliczenia.
To utrudnia masowe sprawdzanie haseł po wycieku bazy.

Jak działa bcrypt? =>

Jak działa bcrypt?

Rejestracja

hasło użytkownika
↓
`bcrypt.hash(hasło, 12)`
↓
hash z unikalnym salt
↓
kolumna password w SQLite

Salt jest zapisany wewnątrz hasha bcrypt.
Nie trzeba tworzyć osobnej kolumny **salt**.
Przykład kodu:

```
const hash = await bcrypt.hash(password, 12);
```

Logowanie

hasło z formularza + hash z bazy
↓
`bcrypt.compare(...)`
↓
true / false

12 to koszt obliczeniowy bcrypta, nazywany też cost factor albo salt rounds.

Im wyższa liczba:

- tym wolniej tworzy się i sprawdza hash;
- tym trudniej atakującemu sprawdzać ogromną liczbę haseł po wycieku bazy;
- tym większe zużycie CPU przez serwer.

Przykładowo:

10 → około 1024 rund

12 → około 4096 rund

14 → około 16384 rund

Atakujemy!

1. Otwórz w nowej karcie: **<http://localhost:3001/api/debug/users>**
2. Znajdź interesującą Cię osobę i wykradnij jej dane

```
▼ [
  ▼ {
    "id": 1,
    "email": "alice@example.test",
    "password": "alice123",
    "display_name": "Alicja"
  },
  ▼ {
    "id": 3,
    "email": "test2",
    "password": "1234",
    "display_name": "test"
  },
  ▼ {
    "id": 4,
    "email": "aaa",
    "password": "aaa",
    "display_name": "aaa"
  }
]
```

Frontend i API to dwa różne originy

Frontend:

`http://localhost:5173`

API:

`http://localhost:3001`

Mimo że oba adresy działają lokalnie, dla przeglądarki są różnymi originami, ponieważ mają różne porty.

CORS: kto może czytać odpowiedź API?

Dozwolony origin:

`http://localhost:5173`



API odpowiada



Przeglądarka udostępnia odpowiedź Reactowi

Niedozwolony origin:

`http://localhost:8080`



API bez właściwego nagłówka CORS



Przeglądarka blokuje odczyt odpowiedzi

Poprawka: ograniczamy origin

```
app.use(cors({  
  origin: 'http://localhost:5173',  
  methods: ['GET', 'POST'],  
}));
```

Element	Reguła
Origin	tylko http://localhost:5173
Metody	GET, POST
API	nadal musi samodzielnie sprawdzać dostęp

Konfiguracja CORS powinna wynikać z rzeczywistych adresów aplikacji, a nie z listy „na wszelki wypadek”.

Helmet: bezpieczniejsze nagłówki HTTP

```
import helmet from 'helmet';  
  
app.use(helmet());
```

X-Content-Type-Options: nosniff

Każe przeglądarce nie „zgadywać” typu pliku. Jeśli serwer deklaruje JavaScript jako zwykły tekst, przeglądarka nie powinna go mimo to uruchomić.

X-Frame-Options: DENY lub SAMEORIGIN

Ogranicza osadzanie strony w <iframe>. Chroni m.in. przed clickjackingiem — nakłonieniem użytkownika do kliknięcia ukrytego elementu banku.

Content-Security-Policy

Określa, skąd strona może ładować skrypty, style, obrazy czy ramki. Jest ważną warstwą ograniczającą skutki XSS, np. może blokować skrypty z obcych domen.

Referrer-Policy

Kontroluje, ile informacji o aktualnym adresie URL przeglądarka wysyła w nagłówku Referer przy przejściu na inną stronę. Pomaga nie ujawniać ścieżek i parametrów adresu innym serwisom.

Błąd nie może być instrukcją dla atakującego

Błędna odpowiedź

```
{  
  "error": "UNIQUE constraint failed: users.email",  
  "stack": "SqliteError: UNIQUE constraint failed..."  
}
```

Bezpieczna odpowiedź

```
{  
  "error": "Nie udało się przetworzyć żądania."  
}
```

Użytkownik potrzebuje komunikatu. Atakujący nie powinien dostawać informacji o wnętrzu serwera.

Co ujawnia wersja startowa?

Kod z server/src/middleware/error-handler.middleware.js:

```
res.status(500).json({  
  error: error.message,  
  stack: error.stack,  
});
```

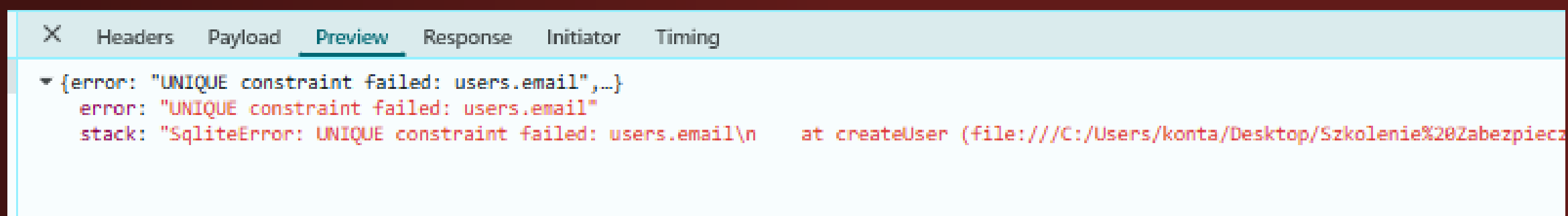
- nazwa i typ bazy danych
- struktura ograniczeń tabel
- ścieżki plików i fragmenty kodu

Atakujemy? =>

Demonstracja: błąd unikalności

1. Otwórz #/access
2. Rejestracja
3. E-mail: alice@example.test
4. Hasło: dowolne-poprawne-hasło
5. Kliknij „Utwórz konto”

API próbuje zapisać istniejący e-mail.
SQLite zgłasza błąd UNIQUE constraint.
Handler przesyła szczegóły do klienta.



The screenshot shows a web browser's developer console with the 'Preview' tab selected. The console displays a JSON error response: `{error: "UNIQUE constraint failed: users.email",...}`. Below this, the raw response is shown in red text: `error: "UNIQUE constraint failed: users.email"` and `stack: "SqliteError: UNIQUE constraint failed: users.email\n at createUser (file:///C:/Users/konta/Desktop/Szkolenie%20Zabezpiecz...`.

Poprawka: szczegóły do logu, minimum do klienta

```
export function errorHandler(error, req, res, next) {  
  console.error(error);  
  
  res.status(500).json({  
    error: "Wystąpił błąd serwera."  
  });  
}
```

Błędy walidacji mają własny kontrolowany format 400.
Globalny handler obsługuje błędy nieoczekiwane.

Dla walidacji błędu 400 możemy zwracać format:

```
{  
  "error": "Dane formularza są nieprawidłowe.",  
  "fields": {  
    "displayName": "Podaj nazwę wyświetlaną.",  
    "email": "Podaj poprawny adres e-mail.",  
    "password": "Hasło musi mieć co najmniej 12 znaków."  
  }  
}
```

To jest dobry format, bo frontend może przypisać każdy komunikat do konkretnego pola:

`errors.email`

`errors.password`

Konfiguracja nie należy do kodu

Źle

```
const JWT_SECRET = "moj-sekret";  
const PORT = 3001;
```

Sekret trafia do repozytorium.
Każdy z dostępem do kodu może go zobaczyć.
Zmiana wymaga edycji i wdrożenia kodu.

Dobrze

```
.env  
PORT=3001  
JWT_SECRET=...
```

process.env.JWT_SECRET (w kodzie)

Kod opisuje zachowanie aplikacji. Środowisko dostarcza konfigurację i sekrety. Każdy programista tworzy własny server/.env na podstawie .env.example.

.env

Lokalna konfiguracja.
Może zawierać sekrety.
Nie trafia do Git.

.env.example

Szablon dla zespołu.
Zawiera nazwy zmiennych.
Nie zawiera prawdziwych sekretów.

.gitignore

Dodajemy .env

Brute force: tysiące zgadywanych haseł

Oś czasu:

10:00:01 błędne hasło
10:00:02 błędne hasło
10:00:03 błędne hasło
10:00:04 błędne hasło
...

Bez limitu:

API obsługuje każdą próbę.
Atakujący może automatycznie zgadywać hasła.
Nawet bcrypt nie wystarcza samodzielnie.
Spowalnia pojedynczą próbę, ale nie blokuje ich liczby.

Rate limiting na trasie logowania:

```
export const authRateLimit = rateLimit({
  windowMs: 15 * 60 * 1000,
  limit: 5,
  standardHeaders: "draft-8",
  legacyHeaders: false,
  message: {
    error: "Zbyt wiele prób logowania. Spróbuj ponownie później."
  }
});
```

Opcja	Znaczenie
windowMs	okres obserwacji: 15 minut
limit	maksymalnie 5 prób
standardHeaders	informacje o limicie w nagłówkach HTTP
message	bezpieczna odpowiedź dla klienta

XSS: kod atakującego w przeglądarce użytkownika

Atakujący zapisuje treść w bazie
(np wiadomości)



Aplikacja renderuje ją jako HTML



Przeglądarka wykonuje obcy JavaScript

XSS atakuje użytkownika aplikacji, nie bazę danych bezpośrednio.

Tekst Reacta i HTML to nie to samo

Bezpieczne:

```
<p>{post.body}</p>
```

Niebezpieczne:

```
<p dangerouslySetInnerHTML={{ __html: post.body }} />
```

React escapuje tekst domyślnie. Ryzyko pojawia się przy świadomym renderowaniu HTML.

Gdy HTML jest wymagany: DOMPurify

```
const safeHtml = DOMPurify.sanitize(html, {  
  ALLOWED_TAGS: ["p", "br", "strong", "em", "ul", "ol", "li", "a"],  
  ALLOWED_ATTR: ["href", "target", "rel"],  
});  
  
return <div dangerouslySetInnerHTML={{ __html: safeHtml }} />;
```

DOMPurify usuwa niebezpieczne tagi i atrybuty, np. script oraz onerror.

Passport: po co jest w tym projekcie?

- Passport jest middlewarem Expressa do uwierzytelniania.
- `passport-local` sprawdza e-mail i hasło w naszej bazie danych
- Passport nie przechowuje użytkowników i nie haszuje haseł. Te zadania pozostają po stronie modeli, SQLite i `bcrypt`
- Po poprawnym logowaniu Passport zapisuje identyfikator użytkownika w sesji

Schemat:

formularz logowania → `POST /api/auth/login` → strategia Local → `bcrypt.compare()` → sesja → cookie `HttpOnly`

Passport-local:

1. Szuka użytkownika po e-mailu w lokalnej bazie SQLite.
2. Porównuje wpisane hasło z hashem przez `bcrypt.compare()`.
3. Jeśli dane są poprawne, Passport tworzy sesję użytkownika.

Strategia Local: gdzie sprawdzamy dane?

Plik: `server/src/config/passport.js`

```
passport.use(new LocalStrategy({ usernameField: 'email' }, async
(email, password, done) => {
  const user = findUserByEmail(email);
  const valid = user && await bcrypt.compare(password, user.password);
  if (!valid) return done(null, false);
  return done(null, { id: user.id, email: user.email });
}));
```

- `usernameField: 'email'` mówi strategii, z którego pola odczytać identyfikator.
- `done(null, false)` oznacza nieudane logowanie bez ujawniania, który element danych był błędny.
- Sukces zwraca minimalny obiekt użytkownika — bez hasha hasła.

Sesja i cookie: co trafia do przeglądarki?

Plik: `server/src/config/passport.js`

```
app.use(session({
  secret: env.sessionSecret,
  resave: false,
  saveUninitialized: false,
  cookie: { httpOnly: true, sameSite: 'lax', secure: false },
}));
```

- Cookie zawiera identyfikator sesji, nie hasło i nie cały obiekt użytkownika.
- `httpOnly: true` blokuje dostęp JavaScriptu do cookie.
- `sameSite: 'lax'` ogranicza wysyłanie cookie w żądaniach cross-site.
- `secure: false` jest tylko dla lokalnego HTTP; na HTTPS w produkcji ustawiamy `true`.

sameSite: lax → Więcej

sameSite: lax

Przykład:

Jesteś zalogowany na bank.local. Twoja przeglądarka ma cookie sesyjne banku. Potem wchodzisz na złośliwą stronę atakujacy.example, która próbuje wysłać żądanie do bank.local.

Bez ograniczeń przeglądarka mogłaby dołączyć cookie banku do takiego żądania. Atakujący mógłby wykorzystać Twoją aktywną sesję, np. do wysłania przelewu — to przykład ataku CSRF.

sameSite: 'lax' powoduje, że cookie jest zwykle wysyłane tylko w kontekście własnej strony. Dopuszcza jednak część bezpieczniejszych przypadków, np. zwykłe wejście przez link (GET) z innej strony. Nie dołącza go natomiast do większości żądań zmieniających dane, takich jak cross-site POST

Przykład kodu atakującego:

```
<form action="https://bank.local/api/transfers" method="POST">
  <input type="hidden" name="recipient" value="konto-atakujacego">
  <input type="hidden" name="amount" value="1000">
</form>

<script>
  document.forms[0].submit();
</script>
```

Ofiara jest zalogowana w bank.local, więc bez ochrony przeglądarka wysłałaby żądanie wraz z cookie sesji banku. Serwer widziałby aktywną sesję i mógłby wykonać operację. Atakujący wykorzystuje fakt, że przeglądarka ofiary zna cookie i może je automatycznie dołączyć.

Ochrona operacji po zalogowaniu

```
export function requireAuthentication(req, res, next) {  
  if (req.isAuthenticated()) return next();  
  return res.status(401).json({ error: 'Zaloguj się, aby wykonać tę operację.' });  
}
```

- `POST /api/posts` wymaga teraz aktywnej sesji.
- Po logowaniu klient wysyła cookie dzięki `credentials: 'include'`.
- Przed logowaniem wywołaj `POST /api/posts`: wynik `401`.
- Po logowaniu tym samym kontem: wpis zostanie przypisany do `req.user.id`.

OAuth 2.0: czym jest i czym nie jest?

Znany przykład: przycisk „Zaloguj przez Google”.

1. Użytkownik klika przycisk w naszej aplikacji bankowej.
2. Przeglądarka przechodzi na stronę Google.
3. Użytkownik podaje hasło wyłącznie Google i zgadza się udostępnić podstawowe dane profilu.
4. Google potwierdza naszej aplikacji, kim jest użytkownik. Nasza aplikacja zakłada własną sesję.

OAuth 2.0 jest mechanizmem bezpiecznego przekazania takiej zgody między aplikacją a zewnętrznym dostawcą. Nasza aplikacja nie poznaje i nie przechowuje hasła do Google.

Inne przykłady: „Połącz aplikację z GitHubem”, „Zezwól aplikacji na dostęp do kalendarza Google”, „Zaloguj przez Microsoft”.

Przykład: logowanie przez Google w Expressie

Zapraszam do pliku PDF



STACJA.IT

**Dziękuję za uwagę!
Do zobaczenia na
kolejnych warsztatach!**

facebook.com/StacjaIT 

linkedin.com/showcase/stacja-it 

PARTNERZY **sages** kodo/amacz
by sages



www.stacja.it